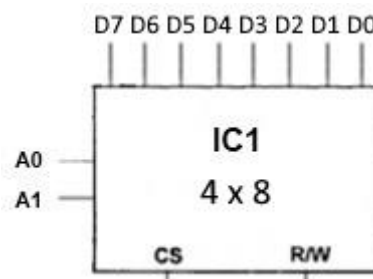


Parte Teórica

- 1** Considere uma máquina com uma cache de mapeamento direto e blocos de 1 Byte com capacidade de armazenar 32KB de dados. A capacidade total desta cache, considerando também os bits da *tag* mais os *valid bits* é de 72KB.
- a) Qual é o espaço de endereçamento da RAM? Justifique. **(1 Val)**
- b) Qual é o número de bits da *tag* e qual a sua função? Justifique. **(2 Val)**

- 2** Considere o circuito integrado de memória RAM da figura, onde A_1, A_0 representam linhas de endereço, $D_7, D_6, D_5, D_4, D_3, D_2, D_1, D_0$ representam linhas de dados, R/W representa a linha de leitura/escrita e CS a linha de *Chip Select*.

Pretende-se dimensionar uma memória capaz de armazenar 30 bytes. Faça um esboço, associando múltiplos circuitos integrados do tipo IC1, de forma a obter a memória RAM pretendida. Deverá ser indicada explicitamente a linha de CS (*Chip Selection*) da memória resultante. **(2 Val)**



- 3** As tecnologias de memória utilizadas em dispositivos de armazenamento secundário apresentam uma evolução considerável das suas características mais relevantes desde o aparecimento dos primeiros computadores até ao dia de hoje. O armazenamento em cartão e papel perfurado, passando pela gravação magnética em fita e discos, bem como o armazenamento ótico são as tecnologias mais antigas. Mais recentemente o armazenamento de estado sólido (Memória **Flash**) adquiriu uma importância crescente. Faça uma descrição detalhada do processo físico que permite representar informação binária através da memória **Flash**, descrevendo aos processos de leitura e escrita. Deve ainda descrever as variantes SLC (Single-Level Cell) e MLC (Multi-Level Cell) deste tipo de memória. **(2 Val)**

Parte Prática

1. Faça um programa em *Assembly*, que permita calcular a média das notas dos alunos, tendo em consideração que o vetor **DADOS**, composto por caracteres, que especificam o nome, seguido de valores numéricos entre 0 e 20, que especificam as respetivas notas. Considere que existe pelo menos uma nota por cada aluno, no máximo 12, e que a quantidade de notas lançadas varia entre os alunos. O vetor **DADOS** termina com caracter zero ("0").

O nome dos alunos e as respetivas médias deverão ser colocadas no vetor **MEDIAS**.

EXEMPLO:

DADOS = "ZE",12,11,10,9,13,"MARIA",19,17,18,"MANUEL",14,16,"0"

MEDIAS = "ZE",11,"MARIA",18,"MANUEL",15

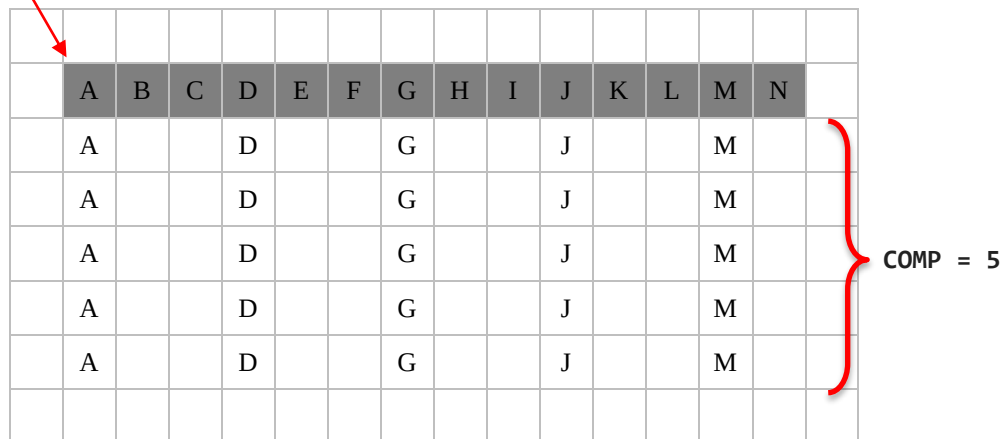
NOTAS:

- Considere que o nome é constituído apenas por caracteres maiúsculos entre "A" e "Z", cujos códigos ASCII são respectivamente 65 e 90. O Código ASCII do caracter zero ("0") é 48.

- No cálculo da média pode desprezar a parte fracionaria, considerando apenas a parte inteira do valor calculado. **(2,5 Val.)**

2. Elabore um programa em *Assembly* que desenhe no ecrã (utilizando acesso direto à memória de vídeo) um pente a partir de uma reta horizontal de letras já existente no ecrã, tal como apresentado na figura seguinte. Os dentes devem ter um espaçamento fixo conforme indicado na figura e o seu comprimento é especificado através a variável **COMP**. O número de dentes depende da dimensão da reta de letras horizontal.

(LIN, COL) = (3, 5)



Nota: Neste programa não é permitida a utilização de interrupções para aceder à memória de vídeo. A memória de vídeo, no caso de sistemas policromáticos, tem início na localização B800h:0000h **(2,5 Val.)**

```
.8086
.model small
.stack 2048

DATA_HERE      SEGMENT
...
DATA_HERE      ENDS
CODE_HERE      SEGMENT
    ASSUME CS:CODE_HERE, DS:DATA_HERE
START:mov ax, DATA_HERE
    mov ds, ax
    mov bx,0b800h
    mov es,bx
    ...
    mov ah,4ch
    int 21h
CODE_HERE      ENDS
END START
```

④



Compuin secundar per $K = 1024 = 2^{10}$

(data per 1Byte = 8 bits)

DADOS = 256 Kbits

(2)

A capacidade total da Cache ($CTC = 72 \text{ KBytes} = 72 \times 8 \text{ Kbits}$)

$$CTC = 576 \text{ Kbits}$$

Pela figura verificasse que a capacidade total da cache

$$CTC = VB + TAG + DADOS$$

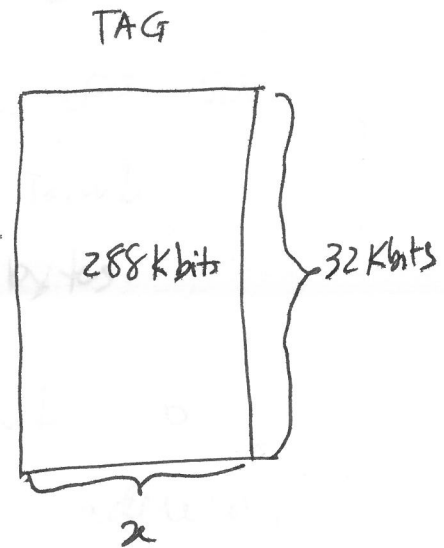
$$576 \text{ Kb} = 32 \text{ Kb} + TAG + 256 \text{ Kb} \Rightarrow TAG = 576 \text{ Kb} - 256 \text{ Kb} - 32 \text{ Kb} = 288 \text{ Kbits}$$

Sendo que o número de bits ocupados pela TAG = 288 Kbits e que aquela 32 Kbits de altura

$$x = \frac{288 \text{ Kbits}}{32 \text{ Kbits}} = 9 \text{ bits}$$

Portanto a TAG tem 9 bits de largura

$$TAG = 9 \text{ bits}$$



A memória RAM um número de bits de Endereço que é

~~End RAM = End Cache~~

$$END_{RAM} = END_{CACHE} + 9$$

$$\text{Como a cache} = 32 \text{ KBytes} = 2^5 \cdot 2^{10} \text{ Bytes} = 2^{15} \text{ Bytes}$$

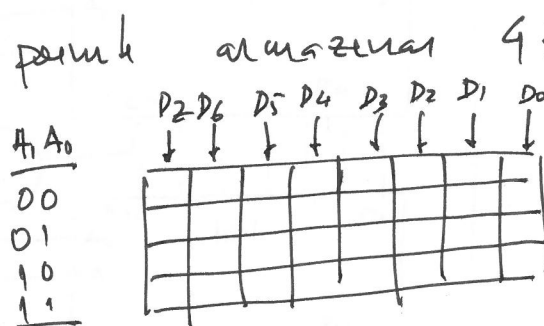
$\Rightarrow$ cache tem 15 ~~bits~~ bits de Endereço logo a RAM

$$END_{RAM} = 15 + 9 = 24 \text{ bits Endereço :}$$

$$\text{Capacidade da RAM} = 2^{24} \text{ B} = 2^4 \cdot 2^{10} \cdot 2^{10} \text{ B} = \underline{\underline{16 \text{ MB}}}$$

(2)

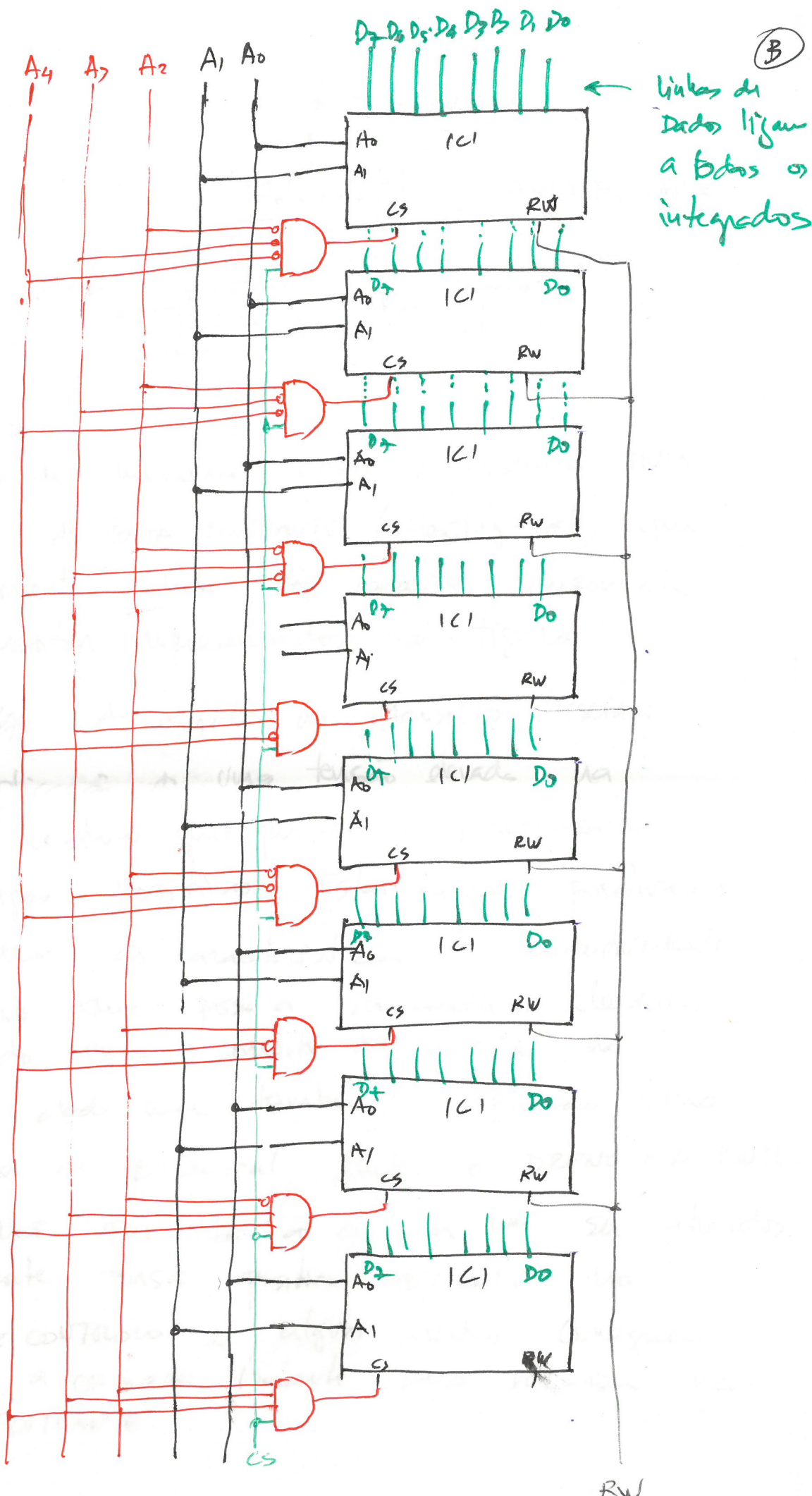
O circuito integrado dado no enunciado possui 2 linhas de endereço A_1 e A_0 . com 2 linhas de endereço é possível endereçar 4 endereços $2^2 = 4$ portanto cada integrado possui 4 Bytes



Como se pretende dimensionar uma memória com capacidade de 32 bytes. significa que será necessário 8 integrados (IC1) que possui a capacidade total de $8 \times 4 = 32$ bytes

Como cada integrado IC1 possui o armazenamento de 1 byte em cada endereço, não é necessário aumentar a largura da palavra.

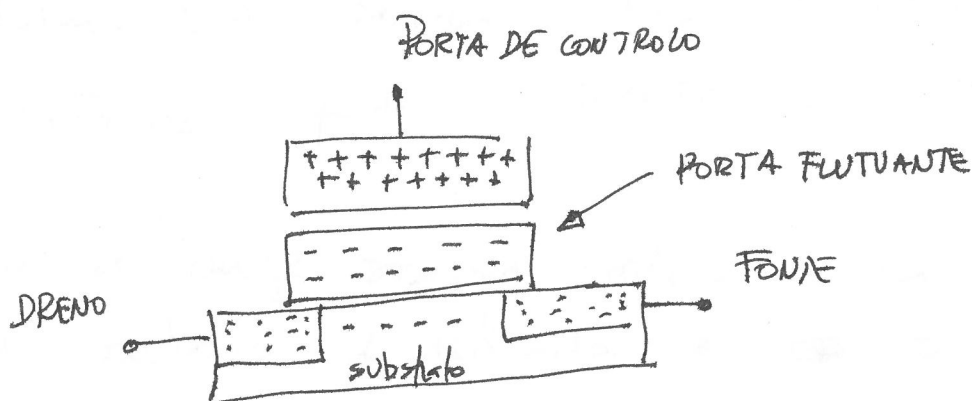
Portanto para armazenar e endereçar 32 células de memória são necessários 5 linhas de endereço ($2^5 = 32$) como a memória disponibilizada possui apenas 2 linhas, são necessários 3 linhas extra ($A_4 A_3 A_2 A_1 A_0$)



③
Linhas de
Dados ligam
a todos os
integrados

RW

3-



A célula de memória FLASH é baseada num transistor de PORTA FLUTUANTE (floating-gate) que é eletricamente isolado dos restantes componentes do transistor representados na figura

~~A escrita no transistor baseia-se~~ A escrita no transistor baseia-se na aplicação de uma tensão elevada na porta de controlo que provoca o aparecimento de cargas positivas. Estas cargas positivas vão alterar as características de condutividade do transistor que passa a ser condutor eléctrico aparecendo uma corrente de eletrões no substituto dado que também é aplicado uma diferença de potencial entre o DRENO e a FONTE. Os eletrões que estão a circular vão ser atraídos pela forte tensão positiva aplicada na PORTA DE CONTROLO e alguns eletrões conseguem vencer a camada isolante para lutarem na PORTA FLUTUANTE.

desta forma vão aparecer uma quantidade de electrões na porta FLUTUANTE.

Entretanto a carga positiva aplicada na porta de controlo é retirada ~~e~~ o que altera as características de condutividade da porta flutuante, que passa instantaneamente de condutor eléctrico para isolante.

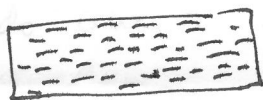
Destá forma os electrões que se encontram na porta FLUTUANTE deixam de se conseguir movimentar e ficam "PREÇOS" nessa zona, mantendo-se por longos períodos de tempo.

Destá forma este transistor ~~é~~ é utilizado como célula de memória FLASH pois permite representar duas situações distintas (0 ou 1) (cargado ou descarregado)

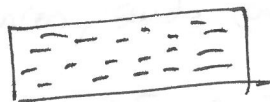
Nesta situação de apenas dois níveis de carga eléctrica podemos dizer que estamos na tecnologia SLC (Single Level Cell) → uma célula grande 1 BIT

no entanto as tecnologias mais avançadas utilizam um codificação de memória

MLC (Multi Level Cell) onde é possível representar vários bits numa única célula por é possível representar vários níveis de eletrões. por exemplo 2 bits



bit ₁	bit ₀
0	0



0	1
---	---



1	0
---	---



1	1
---	---

```
; 'A' - 65  
; 'Z' - 90  
; '0' - 48  
.8086  
.model small  
.stack 2048
```

```
dseg segment para public 'data'
```

```
DADOS db 'ZE',12,11,10,9,13,11,11,11,11,11,'MARIA',19,17,18,'MANUEL',14,16,'0'  
MEDIA db 20 dup (?)
```

```
dseg ends
```

```
cseg segment para public 'code'  
assume cs:cseg, ds:dseg
```

```
main proc  
mov ax, dseg  
mov ds, ax
```

```
xor si, si  
xor di, di
```

```
ciclo:  
cmp DADOS[si], '0'  
je fim
```

```
copia_letras:  
mov al, DADOS[si]  
mov MEDIA[di], al  
inc si  
inc di  
cmp DADOS[si], 'A'  
jae copia_letras
```

```
xor ax, ax  
xor cx, cx
```

```
calculaMedia:  
add al, DADOS[si]  
inc cx  
inc si  
cmp DADOS[si], 20  
jbe calculaMedia  
  
div cl  
mov MEDIA[di], al  
inc di  
jmp ciclo
```

```
fim: mov al, 0  
mov ah, 4ch  
int 21h
```

```
main endp  
cseg ends
```

```
end main
```

P2

```
...
    xor cx, cx
    mov si, 3*160+5*2
ciclo1:
    mov al, es:[si]
    cmp al, 'A'
    jb fim
    cmp al, 'Z'
    ja fim

    push si
    mov cl, comp
ciclo2:
    add si, 160
    mov es:[si], al
    loop ciclo2
    pop si

    add si, 6
    loop ciclo1

fim:
...
```